

REST API Guide for xorkeeSign

Version 1.2

Copyright

All rights reserved.

The Programs, including both the software and documentation, contain proprietary information of Odyssey Technologies Ltd., and they are provided under a license agreement containing restrictions on use and disclosure and are also protected by copyright and other intellectual property law. Reverse engineering of the programs is prohibited.

No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without express written approval of Odyssey.

Odyssey reserves the right to modify specifications cited in this document and to revise the publications without prior notice.

Odyssey does not warrant that this document is error free.

Odyssey has made every effort to ensure the accuracy of its information, as of the date of publication. Because Odyssey must respond to changing market and technological conditions the information herein should not be interpreted as a commitment of any form.

It shall be the licensee's responsibility to take all appropriate fail-safe, back up, redundancy and other measures to ensure the safe use of such applications if the Programs are used for such purposes, and Odyssey disclaims liability for any damages caused by such use of the Programs.

Trademarks

Xorkee and xorkeesign are registered trademarks of Odyssey Technologies Limited. All other company or product names are mentioned for identification purposes only, and may be trademarks of their respective owners. Odyssey disclaims proprietary interests in trademarks and names of others.

Contents

Abstract.....	5
Introduction.....	5
Resources.....	6
Authentication API.....	6
1. Login.....	6
2. Configure an application key.....	9
User Management API.....	11
1. Add a user to the application	11
2. Remove a user	12
Signing API.....	15
1. Get application profiles.....	15
2. PDF signatures – Normal cases.....	17
3. XML signatures – Normal cases	20
4. Customs EDI signatures – Normal cases	22
5. CMS signatures – Normal cases	24
6. Sign data digest.....	27
7. PDF signatures - Immediate	29
8. XML signatures - Immediate	32
9. Customs EDI signatures - Immediate	34
10. CMS signatures - Immediate	36
11. PDF signatures – Guest users.....	38
12. XML signatures – Guest users.....	41
13. Customs EDI signatures – Guest users.....	44
14. CMS signatures – Guest users	46
Document Management API	49
1. Get a document.....	49
2. Delete a document	50
3. Get list of documents for a signer	51
Certificate Issuance API.....	54
1. Get public key	54
2. Sign a CSR digest from any xorkee user	55
3. Deliver certificate	57
Appendix.....	60
a. Application Category.....	60
b. Guest request behavior.....	60
c. Request Parameters	61

d. Status Codes	62
e. Application Error Codes	64
Contact.....	66

Abstract

xorkeesign permits enrolled applications to access many of the features of the service using a simple Application Programming Interface (API). This document describes the calls available in the API in detail.

xorkeesign API follows REST conventions and can be accessed by applications written in most languages that sport a HTTP client.

Introduction

An application can consume the services of xorkeesign by integrating the API. This application should be configured in xorkeesign by a valid user of xorkeesign who is termed as the 'owner' of the application. An application name should be provided by the owner and an application ID is allotted during configuration. The application name and ID are required to login through API.

Subscription:

In addition to the owner's subscription, the API usage itself needs to be subscribed separately. Prior to configuring an Application, the intending owner should ensure that she has adequate subscription units that can be apportioned to the API.

Usage models:

The broad purpose of the API is to enable applications to obtain signatures of xorkee users on different kinds of documents. It may be used by an enterprise application to offer a signed workflow for the staff members; a government department application to present various documents for its officers to sign; a patient record maintenance application to let the medical personnel sign lab results and prescriptions; a law office to obtain the clients' signatures on legal filings; a Certifying Authority to get the subscriber sign CSR & download certificate, and so on.

API access rights:

xorkeesign imposes restrictions on the usage of some APIs based on the application. The applications are categorized into three types based on the application's needs and subject to the owner's agreement to the corresponding terms. These are detailed in the section [Application Category](#) in the APPENDIX.

Application-Signer relationship:

xorkeesign is a digital signing service essentially meant for human users to sign documents after due consideration and not for automated bulk signing applications. An application using the xorkeesign API can pre-associate users with itself by obtaining their consent. When not associated, the application can still get signatures from any xorkee user treating her as a guest if it is registered with sufficient privileges.

Application authentication:

The application using the API (or 'caller' as it would be referred to hereafter) should authenticate with xorkeesign using a login request and a password or a signature conforming to JSON Web Signature

(JWS) standard before calling any of the other requests. On successful authentication xorkeesign returns a *JSON Web Token* (authorization token - JWT) which should be used in all subsequent requests.

The API would be accessed through a server authenticated SSL channel. As the JWS and JWT provide adequate client side authentication and session security, client-authenticated-SSL is not considered necessary and hence not supported in xorkeesign.

Access point:

The API service endpoint shall be

<https://xorkeesign.com/api/>

All the specified API URLs are relative to this base URL.

Resources

Calling Conventions

1. all requests should carry the **Content-Type** as *application/json* and should specify the **Content-Length** whenever the request involves a payload
2. size of the documents fed to xorkeesign through the API cannot exceed 8MB
3. xorkeesign authenticates only the caller application. It does not authenticate any individual users in the course of or through the API

Text conventions used in the document

1. <> indicates that the value between the angular brackets should be provided by the caller at runtime

Authentication API

1. Login

The application connects to xorkeesign specifying its name and the application id allotted at the time of configuration along with the authentication credential as configured in xorkeesign.

On successful authentication xorkeesign returns an authorization token which should be specified in all subsequent requests in the *HTTP Authorization* header as below:

Authorization: Bearer <token>

The returned authorization token would be bound to the IP addresses as specified in the 'clientId' of this API request. This allows the application to use the authorization token from the specified system

IPs.

Application Category

Applications of all categories can use this API.

Request

Method: POST

URL: /auth/login

Body:

If the application is configured for password-based authentication, the following JSON will be sent.

```
{
  'application'      :    <application name>,
  'applicationId'    :    '<application identifier allotted to the application at the time of
configuration>',
  'time'             :    <request time>,
  'validity'         :    <jwt validity>,
  'password'         :    <password>,
  'clientIp'        :    <array of IP list>
}
```

Key	Type	Description
application	String	Required. Descriptive name of the application.
applicationId	String	Required. Application identifier allotted at the time of configuration in xorkeesign.
time	number	Epoch time of the request
validity	number	Optional in seconds. If absent, the value configured in xorkeesign shall be used. If this exceeds the configured value, xorkeesign shall restrict it to the configured value
password	String	Required if password authentication is configured. Base64 encoded password
clientIp	Array of strings	Optional. If present, the API can be accessed from any of the listed IPs. xorkeesign will deny access to other IPs during this session. If absent, the session will be restricted to the IP from which the login request was made.

If the application is configured for key based authentication, a JWS signature created using the private key portion of the key configured for the calling application in xorkeesign. The signature should cover the above JSON as payload should be sent in a single JSON key named 'signature'.

```
{
  signature: <JWS signature>
}
```

JWS signatures are defined in RFC 7515 (<https://datatracker.ietf.org/doc/html/rfc7515>). Open source reference implementations in different languages are available in <https://jwt.io/libraries>

Response

Status Code:

Successful responses will carry a HTTP status code of 200. Errors shall be indicated by one of the codes falling in 4XX and 5XX series of HTTP protocol.

All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'          : 0 | errorCode,
  'token'           : <AUTHTOKEN>
  'validTill'       : session validity for this login
  'errorString'     : "",
  'timestamp'       : ""
}
```

Key	Type	Description
result	number	0 – success other – error code
token	String	Present only on success The authorization token for subsequent requests
validTill	number	Epoch time until which the token can be used
errorString	String	Present only on failure A short description of the error
timestamp	String	Present in the case of failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

The following is the error code specific to this API:

Error Code	Error String	Description
-1083	Incorrect credentials	Incorrect application/owner/password/signature

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

2. Configure an application key

The calling application can change its authentication key or upgrade its authentication method from password to key.

Most applications would store their private and public keys in PEM format with PEM headers and intervening CR/LF pairs. The application should convert the key to a bare base64 format before using it here.

PEM and Base64 formats are defined in RFC 7468 (<https://datatracker.ietf.org/doc/html/rfc7468.html>) and RFC 4648 (<https://www.ietf.org/rfc/rfc4648.txt>) respectively. OpenSSL can be used for PEM to Base64 conversions. Some free online tools are also available for these conversions like https://tomeko.net/online_tools/PEM_to_base64.php?lang=en

Application Category

Applications of all categories can use this API.

Request

Method: PUT

URL: /auth/setkey

Body:

{ signature: <jws signature of the *payload* below computed with the new key>}

Payload:

{

```

    'publicKey' : '<base64 encoded public key in pkcs#1 format>'
    'time'      : '<epoch time of the request>'
  }

```

Response

Status Code:

All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```

{
  result          : 0 | <errorCode>,
  'errorString'   : '<ERRORSTRING>',
  'timestamp'     : ""
}

```

Key	Type	Description
result	Number	0 – success other – failed
errorString	String	Present only on failure A short description of the error
timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1031	Invalid application	Application name is invalid
-1083	Incorrect credentials	Incorrect application/owner/signature

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

User Management API

1. Add a user to the application

Before an application can send signing requests to a user, that user has to be added through this call. A user can be added if she has a valid xorkee handle and is already enrolled in xorkeesign. If the user is not enrolled in xorkeesign, she will be first enrolled in xorkeesign and then added to the application.

Enrollment into xorkeesign requires user consent by way of a signature and hence the application should ensure that the user is available to attend to the notification. User addition to the application requires an additional signed user consent.

Application Category

Applications of all categories can use this API.

Request

Method: POST

URL: /user?userCert=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'      :      '<xorkee handle of the user>',
    'username'    :      '<name of the user>'
}
```

Key	Type	Description
handle	string	Required. User handle as registered in the xorkee service. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
username	string	Required when this user is not enrolled in xorkeesign. This field is ignored otherwise. This name will be used to form an ink-signature image to represent her PDF signatures by default. Hence it is recommended to provide the full name of the user. Username should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'      : 0 | errorCode,
  'errorString' : "",
  'timestamp'   : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure A short description of the error
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1002	User already exists	User already exists in the application
-1008	Username too long	Username length should not exceed 64 characters.
-1015	Declined to enroll	User has declined the enrolment request

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

2. Remove a user

Removes the user's enrolment in the calling application. The application cannot send signature requests to the user any more. However the user can continue to use xorkeesign to sign documents through other interfaces.

The user will not be informed of his removal from the application in this call.

Application Category

Applications of all categories can use this API.

Request

Method: DELETE

URL: /user

Body:

```
{
    'handle'      :      '<xorkee handle of the user>',
}
```

Key	Type	Description
handle	string	Required. User handle as registered in the xorkee service. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
    'result'          : 0 | errorCode,
    'errorString'     : "",
    'timestamp'       :      ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure A short description of the error
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

Signing API

The API calls under this section are used to get a document or digest signed by a user. The signature on the document shall be based on the document type.

xorkeesign can keep the document in its store for a maximum of 3 days from the date of addition. Once the document is signed by the user, xorkeesign restricts the application from deleting the document. This will allow the signer to download the signed document from the xorkeesign interface.

Documents other than PDF documents can be signed with the user's key with or without a digital certificate.

In PDF a digital certificate is mandatory for every signature. In xorkeesign PDF documents can be signed with P-256 ECC keys, even when the signer doesn't have a digital certificate for the key. The PDF signature will be attested by a default signing certificate of xorkeesign.

For increased assurance and branding, organizations may prefer their member's signature to be attested by a certificate issued to the organization rather than the xorkeesign's default certificate. To use such a qualified certificate of an organization, an 'Entity' can be configured in xorkeesign by any xorkeesign user (having a P-256 ECC key) who is termed as the 'admin' of that entity. The admin can associate other users with the entity in a manner similar to adding users in applications.

For a user associated with an Entity, the Entity certificate shall be used instead of the default signing certificate of xorkeesign for attesting her PDF signatures.

xorkeesign offers a signature completion notification facility (**webhook**). To use this feature, the application should be configured with a URL. Whenever a user signs or declines a document that had been previously sent to the user for signature, xorkeesign will return the document or error message to the configured URL. The payload will be same as that of the [getdocument](#) API response. xorkeesign will not reattempt to send the document in case of failures. The application may still be able to retrieve the document by making an explicit 'getdocument' call.

xorkeesign service supports several purpose-specific signature models. Such models include Indian Income tax return signing and Certificate update signatures for Ministry of Corporate Affairs (MCA). These signature formats are supported only through the direct user interfaces in xorkeesign.

The following are a list of API's that can be used to get a document or digest signed by the user:

1. Get application profiles

This request allows the calling application to fetch the profiles configured in xorkeesign. The signature positions and page number are marked as a label(s) in the profile.

Application Category

Applications of all categories can use this API.

Request

Method: GET

URL: /profiles

Body: None

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Header:

Cache-Control: no-cache, no-store, must-revalidate

Body:

```
{
  result          : 0 | <errorCode>,
  profiles        : [Array Of Profiles],
}
```

Where *Profile* is an object as follows,

```
Profile: {
  'name'          : '<profile name >',
  'description'    : '<description>',
  'labels'        : [Array of labels]
}
```

Key	Type	Description
result	number	0 – success other – failed
name	string	Applicable for successful request. Profile name as configured in the xorkeesign web interface.
description	string	Applicable for successful request. Description about the profile.
labels	string	Applicable for successful request. The labels marked for the profiles are returned.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

2. PDF signatures – Normal cases

This API is used to get the PDF documents signed from a user. The user should be pre-associated with the application using the [add user](#) API or through the xorkeesign interface by the application owner.

xorkeesign will get the signature from the user and return the signed document in the response.

If the user is offline xorkeesign will store the document and return a *docId*. The caller can request the user's signature again specifying the *docId* alone and xorkeesign will return the signed document in response as it does for the original request.

The user also on his own can sign such stored documents from his login in xorkeesign. The document shall be returned via the webhook URL. If no webhook is configured or the original request contained the *doNotNotify* flag, the signed document has to be retrieved by the caller through the *getdocument* API.

The signature appearance added to the signed PDF will be taken from the user's preference configured through the xorkeesign interface. This appearance will be used when signatures are requested by any application. The facsimile image in the appearance can be configured/changed through the xorkeesign interface. When no facsimile image is configured, the username provided during enrollment in xorkeesign will be used. The page number and position of the appearance can be controlled through the *pdfSigPage* and *pdfSigPosition* parameters.

Application Category

Applications of all categories can use this API.

Request

Method: POST

URL: /signPdf?userCert=<false>&entity=<false>&doNotNotify=<false>&docId=<number>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'           :    '<signer handle>',
    'document'         :    '<base-64 encoded doc>',
    'filename'         :    '<filename>',
    'fileSize'         :    number,
```

```

    'comments'      :    '<base64 encoded comments >',
    'profile'       :    '<application profile name>',
    'label'         :    '<label name>',
    'pdfSigPageNo'  :    '<page number>',
    'pdfSigPosition':    '[x,y]
}

```

Key	Type	Description
handle	String	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	String	Required. PDF in base64 encoded format. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	String	Required Will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	Number	Optional – in kilobytes Will be displayed to the signer in xorkee app
comments	String	Optional. Shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.
profile	String	Optional. Indicates the application profile to be used for signing the document.
label	String	Required if the profile is specified. The label denotes the position and page number where the signature should be affixed.
pdfSigPage	Number	Currently ignored. Optional. Indicates the page number in which the signature should be affixed. When this is, A positive number indicates the page number starting from the beginning. A negative number indicates the page number starting from the end. For example -1 indicates the last page and so on.

		0 (zero) indicates all the pages. If the number is invalid for the given PDF, this value will default to the last page (-1) If the field is absent a PDF invisible signature shall be affixed.
pdfSigPosition	Array	Currently ignored. Optional. Number array containing the 'x' and 'y' coordinates of the signature appearance. The signature will be centered on this point. If the field is absent a PDF invisible signature shall be affixed.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed PDF>',
  'docId'            : <docId>,
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
Result	Number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	String	Present in failed requests. To be mentioned by the application owner while seeking support for any problem faced with this request.
signedDoc	String	Present only on success. The signed PDF will be returned in base64 encoded format.

docId	Number	Returned when the user is not available to sign. The application should use this in subsequent request to sign or to retrieve the document.
-------	--------	--

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

3. XML signatures – Normal cases

This API is used to get the XML documents signed from a user. The user should be pre-associated with the application using the [add user](#) API or through the xorkeesign interface by the application owner.

xorkeesign will get the signature from the user and return the signed document in the response.

If the user is offline xorkeesign will store the document and return a *docId*. The caller can request the user's signature again specifying the *docId* alone and xorkeesign will return the signed document in response as it does for the original request.

The user also on his own can sign such stored documents from his login in xorkeesign. The document shall be returned via the webhook URL. If no webhook is configured or the original request contained the *doNotNotify* flag the signed document has to be retrieved by the caller through the getdocument API.

Application Category

Applications of all categories can use this API.

Request

Method: POST

URL: /signXml?userCert=<false>&doNotNotify=<false>&docId=<number>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'document'        :    '<base-64 encoded doc>'
```

```

'filename'      :    '<filename>',
'fileSize'      :    number,
'comments'      :    '<base64 encoded comments >'
}

```

Key	Type	Description
handle	string	Required The handle of the signer.
document	string	Required. The XML to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user while signing Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes A original file size can be present even when the request contains just the digest. The size will be displayed to the signer in xorkee app.
comments	string	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```

{
    'result'          : 0 | <errorCode>,
    'signedDoc'       : '<base64 encoded signed XML>',
    'docId'           : <docId>,
    'errorString'     : '<ERRORSTRING>',
    'timestamp'       : ""
}

```

}

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed XML will be returned in base64 encoded format.
docId	number	Present for all cases where the document is stored in xorkeesign. Unique identifier of the stored document. The application should preserve this for later requests to sign or to retrieve the details of the document.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

4. Customs EDI signatures – Normal cases

This API is used to get the India Customs EDI documents signed from a user. The user should be pre-associated with the application using the [add user](#) API or through the xorkeesign interface by the application owner.

xorkeesign will get the signature from the user and return the signed document in the response.

If the user is offline xorkeesign will store the document and return a *docId*. The caller can request the user's signature again specifying the *docId* alone and xorkeesign will return the signed document in response as it does for the original request.

The user also on his own can sign such stored documents from his login in xorkeesign. The document shall be returned via the webhook URL. If no webhook is configured or the original request contained the *doNotNotify* flag the signed document has to be retrieved by the caller through the *getdocument* API.

Application Category

Applications of all categories can use this API.

Request

Method: POST

URL: /signCustomsEdi?doNotNotify=<false>&docId=<number>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'document'        :    '<base-64 encoded doc>',
    'filename'        :    '<filename>',
    'fileSize'        :    number,
    'comments'        :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	string	Required. The Customs EDI document to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes A original file size can be present even when the request contains just the digest. The size will be displayed to the signer in xorkee app.
comments	string	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
    'result'           : 0 | <errorCode>,
    'signedDoc'        : '<base64 encoded signed Customs EDI>',
    'docId'            : <docId>,
    'errorString'       : '<ERRORSTRING>',
    'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed Customs EDI document will be returned in base64 encoded format.
docId	number	Present for all cases where the document is stored in xorkeesign. Unique identifier of the stored document. The application should preserve this for later requests to sign or to retrieve the details of the document.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

5. CMS signatures – Normal cases

This API is used to get any document signed from a user. The user should be associated with the application beforehand using the [add user](#) API or through the xorkeesign interface by the application owner.

The document will be signed in CMS format with the document embedded into the signature. The relying party can verify the signatures and extract the document using xorkee Wand. CMS signatures are standard compliant and can be verified in many compliant applications like OpenSSL.

xorkeesign will attempt to get the signature from the user promptly. The signed document will be immediately returned to the caller and the webhook will not be invoked.

If the user is offline, the caller can request signatures again using the *docId* alone. The webhook will be invoked based on the configuration and the *doNotNotify* flag in the initial request.

Application Category

Applications of all categories can use this API.

Request

Method: POST

URL: /signDocument?userCert=<false>&doNotNotify=<false>&docId=<number>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'document'        :    '<base-64 encoded doc>',
    'filename'        :    '<filename>',
    'fileSize'        :    number,
    'comments'        :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	string	Required The handle of the signer.
document	string	Required. The document to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user

		while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes A original file size can be present even when the request contains just the digest. The size will be displayed to the signer in xorkee app.
comments	string	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
    'result'           : 0 | <errorCode>,
    'signedDoc'        : '<base64 encoded signed document>',
    'docId'            : <docId>,
    'errorString'       : '<ERRORSTRING>',
    'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	String	Present only on failure. A short description of the error.
timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	String	Present only on success. Base64 encoded CMS signature containing the document
docId	number	Present for all cases where the document is stored in xorkeesign. Unique identifier of the stored document. The application should preserve this for later requests to sign or to retrieve the details of

the document.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

6. Sign data digest

The caller can use this API when it doesn't want to share the document with xorkeesign and is capable of carrying out all preparatory processing related to signatures. xorkeesign here is used for the limited purpose of reaching out to the signer and facilitating the signing. In this case, xorkeesign returns the signature from the user as a pkcs#1 or CMS object as specified in the 'pkcs1' query parameter.

As the user will not be able to see the actual content being signed, the caller should provide sufficient details in the 'displayData' field to enable the user to decide if she should sign the digest.

xorkeesign will return failure if the signer is not available at the time of the call. It will also not store the digest with itself. The caller should repeat the request to get it signed.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signDigest?userCert=<false>&pkcs1=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'digest'          :    '<base-64 encoded digest to be signed>',
    'displayData'     :    {} | "
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.

digest	string	Required. Base-64 encoded data digest. The digest algorithm will be inferred from the length of the digest. Supported algorithms: SHA1 (20 bytes) and SHA256 (32 bytes)
displayData	Object or String	Required Used by the Application to inform the details of the data to the signer. The display data can be a JSON object or a string which will be presented to the signer in the xorkee app. JSON objects will be rendered in key-value format in a two column table. String length should be limited to 1024 bytes. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signature'        : '<base64 encoded signature of the given digest>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
Result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
Signature	string	Present only on success. Base64 encoded signature of the digest given in the request.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-18	Signer license expired	The signer's license in Xorkeesign has expired
-1035	Invalid digest algorithm	The provided digest is invalid

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

7. PDF signatures - Immediate

This API is similar to the [signPdf](#) API but xorkeesign will not store the document if the user is not available to sign the document. The caller should repeat the request at a later time to complete the signature.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signPdfImmediate?userCert=<false>&entity=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'document'        :    '<base-64 encoded doc>',
    'filename'         :    '<filename>',
    'fileSize'         :    '<number>',
    'comments'         :    '<base64 encoded comments >',
    'profile'          :    '<application profile name>',
    'label'            :    '<label name>',
    'pdfSigPageNo'     :    '<page number>',
    'pdfSigPosition'   :    '[x,y]'
}
```

Key	Type	Description
handle	string	Required

		The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	String	Required. The PDF to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	String	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes A original file size can be present even when the request contains just the digest. The size will be displayed to the signer in xorkee app.
comments	String	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.
profile	String	Optional. Indicates the application profile to be used for signing the document.
label	String	Required if the profile is specified. The label denotes the position and page number where the signature should be affixed.
pdfSigPage	number	Currently ignored. Optional. Indicates the page number in which the signature should be affixed. When this is, A positive number indicates the page number starting from the beginning. A negative number indicates the page number starting from the end. For example -1 indicates the last page and so on. 0 (zero) indicates all the pages.

		If the number is invalid for the given PDF, this value will default to the last page (-1) If the field is absent a PDF invisible signature shall be affixed.
pdfSigPosition	Array	Currently ignored. Optional. Applicable only for PDF documents. Number array containing the 'x' and 'y' coordinates of the signature appearance. The signature will be centred on this point. This field is considered only if 'pdfSigPage' is present. The preferred signature appearance of the user for the given attestation certificate shall be applied to the PDF document in the specified position and page.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed PDF>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	String	Present only on failure. A short description of the error.
timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	String	Present only on success. The signed PDF will be returned in base64 encoded format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1011	Invalid document	Invalid PDF

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

8. XML signatures - Immediate

This API is similar to the [signXml](#) API but will not store the document when the user is not available (offline) to immediately sign the document.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signXmlImmediate?userCert=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'           :    '<signer handle>',
    'document'         :    '<base-64 encoded doc>',
    'filename'         :    '<filename>',
    'fileSize'         :    number,
    'comments'         :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	String	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.

document	String	Required. The XML to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	String	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The original file size can be present even when the request contains just the digest. The size will be displayed to the signer in xorkee app.
comments	String	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed XML>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	String	Present only on failure. A short description of the error.

timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	String	Present only on success. The signed XML will be returned in base64 encoded format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1012	Invalid document	Invalid XML

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

9. Customs EDI signatures - Immediate

This API is similar to the [signCustomsEdi](#) API but will not store the document when the user is not available (offline) to immediately sign the document.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signCustomsEdiImmediate

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'document'        :    '<base-64 encoded doc>',
    'filename'        :    '<filename>',
    'fileSize'        :    number,
    'comments'        :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	String	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	String	Required. The Customs EDI document to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	String	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The file size will be displayed to the signer in xorkee app.
comments	String	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed Customs EDI document>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	String	Present only on failure. A short description of the error.
timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	String	Present only on success. The signed Customs EDI document will be returned in base64 encoded format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1065	Invalid document	Invalid Customs EDI document

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

10. CMS signatures - Immediate

This API is similar to the [signDocument](#) API but will not store the document when the user is not available (offline) to immediately sign the document.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signDocumentImmediate?userCert=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
```

```

    'document'      :    '<base-64 encoded doc>',
    'filename'      :    '<filename>',
    'fileSize'      :    number,
    'comments'      :    '<base64 encoded comments >'
}

```

Key	Type	Description
handle	String	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	String	Required. The document to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	String	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The file size will be displayed to the signer in xorkee app.
comments	String	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```

{
    'result'          : 0 | <errorCode>,
    'signedDoc'       : '<base64 encoded signed document>',
    'errorString'     : '<ERRORSTRING>',
    'timestamp'       : ""
}

```

}

Key	Type	Description
result	number	0 – success other – failed
errorString	String	Present only on failure. A short description of the error.
timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed document will be returned in base64 encoded format.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

11. PDF signatures – Guest users

This API is similar to the [signPdfImmediate](#) API but the application can request signatures from any xorkee user including ones not pre-associated with the application or even not enrolled with xorkeesign.

This API can be used by applications to get one time signature on document even before onboarding its clients, like loan application form, consent form, etc.

Frequent and sequential declines from guest users can result in the caller being temporarily blocked from xorkeesign. Such blocks will be automatically released after a specific interval but frequent blocking can also result in the caller losing its privilege and in extreme cases the removal of the application from xorkeesign.

This blocking feature exists to protect xorkee users from applications that make unsolicited offers and obtain signatures on questionable premises.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signGuestPdf?userCert=<false>&entity=<false>&store=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'           :    '<signer handle>',

    'document'         :    '<base-64 encoded doc>',
    'filename'         :    '<filename>',
    'fileSize'         :    number,
    'comments'         :    '<base64 encoded comments >',
    'profile'          :    '<application profile name>',
    'label'            :    '<label name>',
    'pdfSigPageNo'     :    <page number>,
    'pdfSigPosition'   :    [x,y]
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	string	Required. The PDF to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The file size will be displayed to the signer in xorkee app.
Comments	string	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.
profile	String	Optional. Indicates the application profile to be used for signing the

		document.
label	String	Required if the profile is specified. The label denotes the position and page number where the signature should be affixed.
pdfSigPage	number	Currently ignored. Optional. Indicates the page number in which the signature should be affixed. When this is, A positive number indicates the page number starting from the beginning. A negative number indicates the page number starting from the end. For example -1 indicates the last page and so on. 0 (zero) indicates all the pages. If the number is invalid for the given PDF, this value will default to the last page (-1) If the field is absent a PDF invisible signature shall be affixed.
pdfSigPosition	Array	Currently ignored. Optional. Applicable only for PDF documents. Number array containing the 'x' and 'y' coordinates of the signature appearance. The signature will be centred on this point. This field is considered only if 'pdfSigPage' is present. The preferred signature appearance of the user for the given attestation certificate shall be applied to the PDF document in the specified position and page.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed PDF>'
```



```

    'errorString'      : '<ERRORSTRING>',
    'timestamp'        : ""
  }

```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed PDF will be returned in base64 encoded format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1011	Invalid document	Invalid PDF

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

12. XML signatures – Guest users

This API is similar to the [signXmlImmediate](#) API but the application can request signatures from any xorkee user including ones not pre-associated with the application or even not enrolled with xorkeesign.

Frequent and sequential declines from guest users can result in the caller being temporarily blocked from xorkeesign. Such blocks will be automatically released after a specific interval but frequent blocking can also result in the caller losing its privilege and in extreme cases the removal of the application from xorkeesign.

This feature exists to protect xorkee users from applications that make unsolicited offers and obtain signatures on questionable premises.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signGuestXml?userCert=<false>&store=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'           :    '<signer handle>',
    'document'         :    '<base-64 encoded doc>',
    'filename'         :    '<filename>',
    'fileSize'         :    number,
    'comments'         :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	string	Required. The XML to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The file size will be displayed to the signer in xorkee app.
comments	string	Optional. Information to be shown to the signer in the xorkee app while signing.

Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed XML>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed XML will be returned in base64 encoded format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1012	Invalid document	Invalid XML

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

13. Customs EDI signatures – Guest users

This API is similar to the API [signCustomsEdiImmediate](#) but the application can request signatures from any xorkee user including ones not pre-associated with the application or even not enrolled with xorkeesign.

Frequent and sequential declines from guest users can result in the caller being temporarily blocked from xorkeesign. Such blocks will be automatically released after a specific interval but frequent blocking can also result in the caller losing its privilege and in extreme cases the removal of the application from xorkeesign.

This feature exists to protect xorkee users from applications that make unsolicited offers and obtain signatures on questionable premises.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signGuestCustomsEdi?store=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',

    'document'        :    '<base-64 encoded Customs EDI document>',
    'filename'         :    '<filename>',
    'fileSize'         :    number,
    'comments'         :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	string	Required. The Customs EDI document to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures.

		The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The file size will be displayed to the signer in xorkee app.
comments	string	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed Customs EDI document>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed Customs EDI document will be returned in base64 encoded format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1065	Invalid document	Invalid Customs EDI document

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

14. CMS signatures – Guest users

This API is similar to the API [signDocumentImmediate](#) but the application can request signatures from any xorkee user including ones not pre-associated with the application or even not enrolled with xorkeesign.

Frequent and sequential declines from guest users can result in the caller being temporarily blocked from xorkeesign. Such blocks will be automatically released after a specific interval but frequent blocking can also result in the caller losing its privilege and in extreme cases the removal of the application from xorkeesign.

This feature exists to protect xorkee users from applications that make unsolicited offers and obtain signatures on questionable premises.

Application Category

Only ID-Validated and Privileged applications are allowed to use this API.

Request

Method: POST

URL: /signGuestDocument?userCert=<false>&store=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'           :    '<signer handle>',
    'document'         :    '<base-64 encoded document>',
    'filename'         :    '<filename>',
    'fileSize'         :    number,
    'comments'         :    '<base64 encoded comments >'
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
document	string	Required. The document to be signed should be provided here in base64 encoded format. xorkeesign uses SHA256 as the digest algorithm for signatures. The document size (before encoding) should not exceed 8 MB. If this is exceeded error will be returned.
filename	string	Required Name of the file to be signed. This will be displayed to the user while signing. Filename should not exceed 256 characters. If this is exceeded error will be returned.
fileSize	number	Optional – in kilobytes The file size will be displayed to the signer in xorkee app.
comments	string	Optional. Information to be shown to the signer in the xorkee app while signing. Comments should not exceed 64 characters. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'signedDoc'        : '<base64 encoded signed document>',
  'errorString'       : '<ERRORSTRING>',
  'timestamp'        : ""
}
```

Key	Type	Description
result	number	0 – success other – failed

errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signedDoc	string	Present only on success. The signed document will be returned in base64 encoded format.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

Document Management API

1. Get a document

This request allows the calling application to fetch a document by specifying the *docId* obtained from an earlier call to xorkeesign.

Application Category

Applications of all categories can use this API.

Request

Method: GET

URL: /document?docid=<docId>

Body: None

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Header:

Cache-Control: no-cache, no-store, must-revalidate

Body:

```
{
  'result'           : 0 | errorCode,
  'document'         : '<base-64 encoded>',
  'docStatus'        : 'signed' | 'toBeSigned' | 'rejected',
  'docType'          : 'pdf' | 'xml' | 'others',
  'comments'         : '<COMMENTS>'
  'errorString'      : '<ERRORSTRING>',
  'timestamp'        : ''
}
```

Key	Type	Description
result	number	0 – success other – failed
document	string	Applicable for successful request.

		The requested document in base64 encoded form.
docStatus	string	Applicable for successful request. Current Status of the document. The possible values are • 'toBeSigned' • 'signed' • 'rejected'
docType	string	Applicable for successful request. Type of the document. Possible values are • 'pdf' • 'xml' • 'others'
comments	string	Optional and available only upon success Comments provided while providing the document to Xorkeesign
errorString	string	Optional and available only upon failure Provides a short description of the error code
timestamp	string	Optional and available only upon failure When provided user can check the system logs for the corresponding timestamp for detailed errors and troubleshooting.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

2. Delete a document

This request allows the calling application to delete a document in xorkeesign before the user signs it by specifying the document id. Only the user can delete a document from her interface once she has signed it.

The document is maintained in xorkeesign storage for 3 days from the day of addition . If not deleted by the user or application, it would be auto deleted irrespective of its status of signing.

Application Category

Applications of all categories can use this API.

Request

Method: DELETE

URL: /document?docid=<docId>

Body: None

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'      : 0 | <errorCode>,
  'errorString' : '<ERRORSTRING>',
  'timestamp'   : ""
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure A short description of the error
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

3. Get list of documents for a signer

This request lists the documents uploaded by this application for a particular signer and currently available in the server. Any of these documents can be fetched by specifying its *docId* in a [getdocument](#) call.

Application Category

Applications of all categories can use this API.

Request

Method: GET

URL: /documents?handle=<handle>

Body: None

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Header:

Cache-Control: no-cache, no-store, must-revalidate

Body:

```
{
  result          : 0 | <errorCode>,
  docs            : [Array Of DocInfo],
  'errorString'   : '<ERRORSTRING>',
  'timestamp'     : ""
}
```

where *DocInfo* is an object as follows,

```
DocInfo: {
  'docid'          : '<DOCID>',
  'filename'       : '<FILENAME>',
  'docType'        : '<DOCTYPE>',
  'docSize'        : '<DOCSIZE>',
  'receivedOn'     : '<UPLOADEDON>',
  'signedOn'       : '<SIGNEDON>',
  'rejectedOn'     : '<REJECTEDON>',
  'comments'       : '<COMMENTS>'
}
```

Key	Type	Description
result	number	0 – success other – failed
docs	<array>	Present if the request was successful. Array of objects. Each object will contain the details of one document.
docId	string	Unique identifier generated by Xorkeesign for each received document.

filename	string	Name of the file
docType	string	Type of the document. Possible value are • 'pdf' • 'xml' • 'others'
docSize	<numeral>	Size of the document in KB.
receivedOn	string	Time (in Epoch format) at which the document was received from the application.
signedOn	string	Time (in Epoch format) at which the document was signed. This field is applicable only for signed documents.
rejectedOn	string	Time (in Epoch format) at which the document was rejected by the signer. This field is applicable only for rejected documents.
comments	string	Comments sent in the request while uploading the document to Xorkeesign.
errorString	string	Present only on failure A short description of the error
timestamp	string	Time in epoch format. Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

Certificate Issuance API

These calls can currently be used only by a Certifying Authority (CA) recognized in xorkee as a qualified CA. Further the application used by the CA should be registered in xorkeesign under the privileged category.

1. Get public key

If the CA is creating the CSR for its subscriber with the subject DN details available, it may fetch the public key from the subscriber using this call. The xorkee handle should be obtained by the caller from the subscriber.

The subscriber need not be a user of xorkeesign for this call.

Application Category

Only privileged applications.

Request

Method: GET

URL: /publickey?handle=<handle>&noToken=<>false>

Body: None

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
  'result'           : 0 | <errorCode>,
  'errorString'      : '<ERRORSTRING>',
  'timestamp'        : '',
  'keyType'          : 'rsa2048'|'eccp256' ,
  'publicKey'        : '<base64 encoded public key of the user>',
  'tokenDetails'     :
  {
    'serialNo'       : <token serial number> ,
    'model'          : <token model>,
    'manufacturer'   : <token manufacturer name>
  }
}
```

Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
keyType	string	Present only on success. The key algorithm of the specified handle is returned.
publicKey	String	Present only on success. The base64 encoded public key is returned in PKCS#1 format.
tokenDetails	Object	Present only for token users upon success. The token serial number, model and manufacturer are returned as string in this object.

Error Codes:

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

2. Sign a CSR digest from any xorkee user

The call can be used by configured CA applications that creates the CSR to get that CSR signed from a subscriber. The call offers additional flags to specify key location and to indicate to xorkeesign that a certificate delivery will follow immediately.

Application Category

Only privileged applications.

Request

Method: POST

URL: /signCsrDigest?noToken=<false>&waitForCert=<false>

The effect of different parameters in query strings is explained in the section [Request Parameters](#).

Body:

```
{
    'handle'          :    '<signer handle>',
    'digest'          :    '<base-64 encoded CSR digest to be signed>',
    'displayData'     :    {} | ""
}
```

Key	Type	Description
handle	string	Required The handle of the signer. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.
digest	string	Required. Base-64 encoded digest of the CSR. The digest algorithm should be SHA256 only (32 bytes).
displayData	string or Object	Required Used by the Application to inform the details of the data to the signer. The display data can be a JSON object or a string which will be presented to the signer in the xorkee app. JSON objects will be rendered in key-value format in a two column table. String length should be limited to 1024 bytes. If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
    'result'          :    0 | <errorCode>,
    'signature'       :    '<base64 encoded signature of the given CSR digest>',
    'errorString'     :    '<ERRORSTRING>',
    'timestamp'       :    ""
}
```


Key	Type	Description
result	number	0 – success other – failed
errorString	string	Present only on failure. A short description of the error.
timestamp	string	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.
signature	string	Present only on success. Base64 encoded signature of the digest given in the request in PKCS#1 format.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1123	Key not in token	User does not have the signing key in a USB crypto token.

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

3. Deliver certificate

This call can be used only by a Certifying Authority (CA) recognized in xorkee as a qualified CA. Moreover the application used by the CA should be configured in xorkeesign under the privileged category.

A CA can use this to deliver a digital certificate issued by it to a subscriber who has a valid xorkee handle and a crypto token with the private key for that digital certificate. It is not required that the certificate subscriber should be a member of xorkeesign.

This eliminates the need for digital certificate subscribers to install complex token drivers and in many cases an additional certificate utility for storing the certificate in the token after the certificate is issued. This also enables users with only an android phone to apply for and obtain qualified digital certificates.

This call requires that the subscriber's crypto-token is plugged into a mobile or PC at the time of the call for the import to happen immediately. If not, the xorkee environment shall hold the certificate until the user plugs in his token into a mobile or PC and then deliver it. (The CA shall not get a status

update on such delayed deliveries as there will be no message path between the xorkee server that does this and xorkeesign or the CA). Regardless of the immediate or delayed delivery of the certificate to the token, the calling CA application will get a success response to this request.

Application Category

Only privileged applications.

Request

Method: POST

URL: /delivercertificate

Body:

```
{
    'certificate'      : <base64 encoded certificate>,
    'handle'          : <user handle>,
}
```

Key	Type	Description
certificate	String	Required. The base64 encoded certificate of the subscriber that is to be imported into the token. Either the handle or the old certificate is required to identify the subscriber.
handle	String	Optional The handle of the subscriber. Handle should not exceed 64 characters (excluding @xorkee.com). If this is exceeded error will be returned.

Response

Status Code: All the codes returned by xorkeesign are listed in the section [Status Codes](#) in the APPENDIX.

Body:

```
{
    'result'          : 0 | <errorCode>,
    'errorString'     : '<ERRORSTRING>',
}
```

```

    'timestamp'      : ""
  }

```

Key	Type	Description
result	number	0 – success other – failed
errorString	String	Present only on failure A short description of the error.
timestamp	String	Present in failed requests. Serves as a pointer to the approximate position in server logs for troubleshooting.

Error Codes:

The following is a list of error codes specific to this API:

Error Code	Error String	Description
-1024	Key mismatch	Public key present in the certificate does not match the user's key associated with xorkee handle
-1098	User declined	User has declined the certificate import request.
-1099	User not reachable	Unable to reach the user for importing the certificate in the token.
-1100	User not ready	Either the user's phone is out of network coverage or she has not plugged the crypto-token in the mobile
-1121	CA certificate not in trusted list	The certificate issuer is not a trusted CA .
-1122	Certificate revocation check failed	Certificate revocation check could not be done.

There could be other application error codes returned by more than one request. Refer section [Application Error Codes](#) in the APPENDIX for such error codes.

Appendix

a. Application Category

The applications configured in xorkeesign will fall in one of these categories based on certain conditions fulfilled and undertakings given by the owner of the application at the time of configuration:

Requirements	Category
1. Digital certificate from a qualified CA 2. Agreement consenting to the terms and conditions relevant to this category is to be executed by the owner with xorkeesign service 3. If it is a CA, signing certificate of the CA should be configured in xorkeesign 4. Approval by the xorkeesign administrator 5. xorkeesign subscription for owner user	Privileged
1. Digital certificate from a qualified CA or identity verified with a eSign CA 2. Agreement consenting to the terms and conditions relevant to this category is to be executed by the owner with xorkeesign service. 3. xorkeesign subscription for owner user	ID-Validated
1. xorkeesign subscription for owner user 2. Agreement relevant to the Standard category	Standard

b. Guest request behavior

The behavior of signGuestPdf, signGuestXml, signGuestCustomsEdi, signGuestDocument calls under various user states is shown in the table below:

Enrolled in xorkeesign	Consented to the API	Signature Request status	Guest user license consumed
Yes	Yes	Successful	No
Yes	No	Successful	Yes
Yes	No	Forbidden	No

No	No	Successful	Yes
No	No	Unknown User	No

c. Request Parameters

The behavior of most APIs can be controlled by the parameters in the query string. The following are a list of parameters and their influence in the API behavior:

Parameter	Possible Values	Description
userCert	true false	xorkeesign honors the request based on the availability of a digital certificate associated with the signer handle. If this parameter is true, the call to perform user association or to sign a document will fail if the user doesn't have a digital certificate. If this parameter value is false (which is the default) or when the parameter is absent, the request will be processed even when the user doesn't have a digital certificate. When the user's key is in a token but doesn't have a digital certificate, xorkeesign will be unable to sign PDF documents.
entity	true false	This parameter is applicable only for PDF documents. When set to false (default), the PDF document signed using the user's key is attested with the certificate of xorkeesign. Otherwise if the user is associated with an Entity in xorkeesign, the attestation is performed using the digital certificate of the Entity. The API will fail if the caller sets the value as "true" for a user who is not mapped with an Entity. It will also fail if it is used with both <i>entity</i> and <i>userCert</i> set to true.
pkcs1	true false	This flag is applicable only for signDigest call. When false (default), the signature will be returned as a CMS object. When set to true the signature will be in PKCS#1 format. If the user has a digital certificate it will be embedded into the CMS object.
doNotNotify	true false	If the signer is not immediately available when one of the sign APIs is called the document will be stored in xorkeesign where

		the signer can login later and sign/reject the file. If a webhook (a return URL) is configured for the application xorkeesign will notify the application whenever the user signs, rejects or deletes the document or the document storage lapses after three days without action from the user. Only the first of these actions will be so notified. If the notification is triggered by the user signing, the application will also receive the signed document in the return URL. If this flag is set to <i>true</i> the notification will not take place regardless of the user action. The application has to make a getdocument call to ascertain the state of the document. If the webhook is not configured this flag has no effect.
docId	number	To sign a document previously stored in xorkeesign. All other fields shall be taken from the original request that caused the document to be stored. When this parameter is present, the other fields/parameters in this request are ignored.
waitForCert	true false	This will be used only by a CA application and defaults to false. The parameter is applicable only when the caller requests the user to sign a CSR digest. If the certificate will be ready immediately after getting the signed CSR, the application can set this to true. This will enhance the user experience by importing the issued certificate in a single user notification. If set to false, the user will have to act upon two different notifications, first to sign the CSR and then to import the issued certificate to the token.
noToken	true false	This will be used only by a CA application during the certificate issuance process. Using this parameter the caller can restrict certificate issuance only if the private key is available in a smart card or an USB crypto token.

d. Status Codes

The following are the possible status codes for any API:

Status	Message	Description
200	OK	Success

		Some possible scenarios are listed below: 1. caller has been authenticated 2. user association with the application has been removed 3. document or digest signed successfully 4. document deleted from xorkeesign
201	Created	Success This indicates any one of the following: 1. upgraded to key authentication 2. authentication key modified 3. user associated with the application 4. document saved in server when the signer is offline
204	No Content	Success 1. document deleted from xorkeesign 2. no documents available in the user's space by xorkeesign
400	Bad Request	Error This can be due to: 1. malformed request 2. insufficient inputs
401	Unauthorized	Error This indicates any one of the following: 1. authentication failed due to invalid credentials 2. the authorization token is missing, has expired or is invalid
403	Forbidden	Error This error is sent in the following scenarios: 1. The caller is not allowed to use this API 2. User declined to enroll in the application 3. User does not belong to the application 4. Document does not belong to the user or caller
404	Not Found	Error This indicates any one of the following: 1. user not associated with the application 2. document not found
405	Method Not Allowed	Error The HTTP request method provided by the caller was different from the expected method.
500	Internal Server Error	Error The response body contains the error description.
503	Service unavailable	Error Service temporarily unavailable. The application should try it again.

e. Application Error Codes

The following are some possible error codes returned by xorkeesign for any API:

Error Code	Error String	Description
-1	System error	Could occur due to hardware/software/connectivity failures in the server.
-2	Invalid request	Unsupported request or malformed request parameters
-3	Incomplete request	Required field(s) in the request are missing
-4	Access denied	Document does not belong to the signer or to the application
-6	xorkee connection failed	Unable to contact xorkee
-14	License expired	The API license/subscription expired and should be renewed.
-15	Handle too long	User handle length (excluding @xorkee.com) should not exceed 64 characters
-16	Filename too long	Length of the filename should not exceed 256 characters
-17	Comments in request too long	Length of the comments should not exceed 64 characters
-18	Signer license expired	The signer's license in xorkeesign has expired
-19	User count exhausted	The API user count has exceeded the subscribed count and should be renewed.
-21	Privilege violation	The application does not have privilege to use the API
-1001	User unknown	Signer handle not enrolled with Xorkeesign
-1004	User not enrolled	User is not associated with the requesting application.
-1010	Unknown docid	The provided docid is not available or is invalid. The docid could have been (auto)/deleted from xorkeesign store.
-1011	Invalid document	Invalid PDF
-1012	Invalid document	Invalid XML
-1014	User declined	User declined to sign the document
-1016	Request expired	User has not responded to the notification
-1019	Base64 decoding failed	The provided base64 encoded data is invalid
-1020	Certificate absent	User doesn't have certificate
-1035	Invalid digest algorithm	The provided digest is invalid
-1042	User not reachable	Unable to reach the user
-1043	User blocked	User handle is blocked in xorkee. Ask a nominee to unblock the same.
-1044	Invalid Handle	Not a valid xorkee handle

-1050	Certificate expired	User certificate has expired
-1051	Certificate revoked	User certificate has been revoked
-1090	Default signing certificate expired	The default signing certificate has expired
-1091	Default signing certificate revoked	The default signing certificate has been revoked
-1102	Missing authorization	Authorization header not found
-1103	Invalid authorization token	The provided token is invalid
-1104	Expired authorization token	The provided token has expired. Application should get a new token using the "Login" API.
-1105	Default signing certificate not configured	The default signing certificate that is required for attestation signature is not configured in the Xorkeesign
-1106	Document size too large	The document size is greater than 8 MB.
-1111	Document already signed	The input document or the stored document pertaining to the docId is already signed by the same user.
-1115	Public key not found	User key could not be retrieved with the specifications imposed by the CA application
-1125	Display data too large	Size of the data to be displayed to the signer in the xorkee app is greater than 1024 bytes.

Contact

Odyssey Technologies Limited

5th Floor, Dowlath Towers

63, Taylors Road, Kilpauk

Chennai – 600 010

INDIA.

For Technical Support:

Email support@odysseytec.com

Phone +91 44 26450082, 26450083

website www.odysseytec.com